

Frontend Engineering

Phases 5–6 · Modern React, TypeScript & Production-Grade UI Engineering

HTML, CSS, Tailwind & shadcn/ui, JavaScript, and TypeScript fundamentals, followed by a modern React 19 + TypeScript application layer — component architecture, forms, data fetching, state management, accessibility, testing, and performance — wired to the real Spring Boot API you built in Part A. Every project extends one continuously evolving ShopFlow codebase; nothing is rebuilt from scratch.

Phases 5–6

15 Topics

8–10 Weeks

Beginner → Intermediate

8–12 hrs/week

A11y & Testing Built In

Phases	5–6 of 10	Topics	15
Estimated Duration	8–10 weeks	Difficulty	Beginner → Intermediate
Suggested Pace	8–12 hrs/week · 2–3 topics/week (part-time default)	Projects	2–4 hands-on builds per topic, all on one live codebase
Documentation Sources	2+ official/primary sources per topic	Continuity	Every project extends the ShopFlow app started in Part A

What Changed in This Revision

Reviewed against professional 2026 frontend practice. **[NEW]** = added topic/subtopic. **[UPDATED]** = deepened, reordered, or replaced. Unmarked topics carry over from the original roadmap. This edition also removes every standalone "toy" project — Todo lists, weather widgets, throwaway portfolios — in favor of building the same skill directly inside ShopFlow, so each topic leaves the product measurably better instead of resetting to a blank slate.

- REPLACED** Bootstrap → **Tailwind CSS & shadcn/ui**. Radix-primitive, copy-in components are the pattern dominating new React libraries, and feed directly into the Design System & Component Library work in Phase 6.
- DEEPENED** **CSS** adds container queries and logical properties — the real answer to component-level responsiveness, since a component rarely knows its own viewport.
- DEEPENED** **JavaScript** adds the event loop (microtasks vs. macrotasks) — the mechanism behind most async bugs students later hit in React.
- DEEPENED** **React + TypeScript** adds **React 19** (Actions, useOptimistic, useActionState, use(), ref-as-prop), useReducer, custom hooks, error boundaries, and SPA focus management.
- ADDED** **Component Architecture, Design Systems & Composition Patterns** — compound components, controlled vs. uncontrolled, design tokens, prop drilling vs. lifting state, taught explicitly.
- ADDED** **Forms with React Hook Form + Zod** — hand-rolled form state is exactly the repetitive work the ecosystem already solved.
- REPLACED** Axios & API Integration → **Axios + TanStack Query**. Axios stays for the client/interceptor layer; caching and background refetching move to React Query.
- ADDED** **State Management: Context vs. Zustand** — separates server state (TanStack Query) from client/UI state, and names when each tool is actually the right one.
- ADDED** **Testing & Accessibility Audits: Vitest, RTL, axe-core & Playwright** — component tests, automated a11y assertions, and one E2E flow, immediately after components are built.
- ADDED** **Performance: Code-Splitting, Suspense & Core Web Vitals** — the same "measure before optimizing" instinct from Part A's database phase, applied to the frontend, with a scored Lighthouse CI gate.
- ADDED** **Next.js Primer** — SSR, file-based routing, Server Components, and structured-data SEO in one guided rebuild of a single ShopFlow page. Deliberately scoped as a primer, not a migration.
- UPDATED** Every phase now closes with a **Ship-It Checklist** (testing, accessibility, debugging, production-readiness) instead of leaving those as an afterthought.

Topics Covered

Phase 5 — Foundations	HTML · CSS (+ Container Queries) · Tailwind CSS & shadcn/ui · JavaScript ES6+ (+ Event Loop) · TypeScript
Phase 6 — Application Layer	React 19 + TypeScript · Component Architecture & Design Systems · Forms (RHF + Zod) · Axios + TanStack Query · State Management (Context vs. Zustand) · Types of APIs · Testing & A11y Audits (Vitest/RTL/axe/Playwright) · Performance & Core Web Vitals · Frontend Dev Tools · Next.js Primer

Next: with a working, tested, accessible, performance-checked React frontend talking to a real backend, move on to **Part C — Deployment, Cloud & Integrations**.

PHASE 5 OF 10

Frontend Foundations

HTML · CSS (+ Container Queries) · Tailwind & shadcn/ui · JavaScript (ES6+) · TypeScript

Before React, get fluent in the languages the browser actually runs. This is what lets you debug a layout bug or a broken event handler without guessing — and what separates someone who can use React from someone who understands it.

Continuity: every project in this phase works directly on the **ShopFlow Storefront** you scaffold in Topic 1 (product listing, product detail, cart) and on a companion **developer portfolio** site used to showcase it. No project starts a new codebase — each one refines the last commit.

HTML

The structural skeleton of every page — semantic, accessible markup that means something, not just divs everywhere.

Subtopics: semantic HTML5 elements · forms & validation attributes · accessibility (ARIA, alt text) · meta tags, Open Graph & SEO basics

1. ShopFlow Skeleton — structure before style

Scaffold product listing, product detail, and cart pages with header/nav/main/section/article/footer → no CSS yet, structure only → validate with the W3C HTML validator → confirm the outline in browser dev tools reads like a table of contents
2. Accessible Checkout & Auth Forms

Add a proper label for every input on the (stubbed) checkout, login, and register forms → use correct input types (email, tel) → add required/pattern attributes → tab through every form using only the keyboard
3. ARIA-Enhanced Cart Drawer

Build the cart drawer as a dropdown/panel → add role, aria-expanded, and aria-live for item-count updates → test with a screen reader or accessibility dev tools → fix every issue the audit flags
4. Developer Portfolio — semantic scaffold

Structure your own portfolio (about, projects, contact) with the same semantic rigor → add Open Graph and meta tags so it previews correctly when shared → this becomes the site that showcases ShopFlow later

Study Resources

- MDN: HTML Basics
- MDN: Learn HTML
- web.dev: Learn HTML

CSS Updated

Layout and visual presentation — box model, modern layout systems, and responsive design, now including component-level responsiveness.

Subtopics: box model & positioning · Flexbox · Grid · media queries · CSS variables & transitions · **container queries [NEW]** · **logical properties [NEW]**

1. ShopFlow Nav + Grid Catalog

Build the storefront navbar with Flexbox, collapsing to a mobile menu via media query → build the product catalog with CSS Grid using auto-fit/auto-fill → reflow the grid from 4 to 2 to 1 columns and confirm at 3 real viewport widths
2. Container Query Product Card

Wrap the existing ProductCard in container-type: inline-size → write breakpoints that switch it from stacked to side-by-side → drop the same card into a narrow sidebar (related items) and the wide main grid → confirm it adapts independently of viewport width
3. Logical Properties Pass

Replace left/right-based spacing and positioning across the storefront with margin-inline, padding-block, etc. → flip <html dir="rtl"> and confirm layout mirrors correctly with zero extra CSS

Study Resources

- MDN: CSS Reference
- CSS-Tricks: A Complete Guide to Flexbox
- MDN: CSS Container Queries

Tailwind CSS & shadcn/ui Updated

Utility-first styling plus the headless-component pattern (Radix primitives + Tailwind) dominating new React libraries — replacing Bootstrap, now a legacy pattern in greenfield React work.

Subtopics: Tailwind utility classes & config · responsive breakpoint prefixes · shadcn/ui: copy-in components, not npm-installed · Radix primitives under the hood · theming with CSS variables

1. Migrate ShopFlow to Tailwind

Rebuild the existing storefront's CSS as Tailwind utility classes, no separate stylesheet → use sm:/md:/lg: prefixes for the breakpoints you already defined → delete the old CSS file once parity is confirmed
2. Design Tokens in Config

Extend tailwind.config with ShopFlow's brand color palette and font family → apply the tokens across the nav, buttons, and cards → confirm the production build only includes classes actually used
3. shadcn/ui Component Pull

Run the shadcn/ui CLI to add Dialog and Select → wire the Dialog into the cart drawer and Select into a sort/filter control → read the generated files — this code now lives in your repo, not node_modules → comment on why "copy the code in" is a different trust model than npm install

4. Bootstrap-for-Comparison (trimmed)

On a throwaway page only, add Bootstrap via CDN and drop in a navbar + modal using prebuilt classes → note how much JS behavior comes bundled versus what Tailwind+shadcn required you to wire yourself → write 2 bullets on when you'd still choose Bootstrap (e.g. an internal admin tool on a tight deadline)

Study Resources

- Tailwind CSS Documentation
- shadcn/ui Documentation
- Radix UI Primitives

JavaScript (ES6+) Updated

The language that runs in the browser — modern syntax, the DOM, and async programming, now including the execution model behind the async bugs you'll hit in React.

Subtopics: let/const, arrow functions, destructuring · DOM manipulation & events · Promises & async/await · closures & modules · array methods (map/filter/reduce) · the event loop: microtasks vs. macrotasks [NEW]

1. ShopFlow Cart in Vanilla JS

Render the cart from a JS array of items → add/remove/adjust-quantity handlers → compute a running total → persist the cart to localStorage so it survives a refresh

2. Live Product Search

Call your real product data (local JSON stand-in for the API) with fetch() as the user types → handle the loading state while waiting → handle the error/empty-results state → render matching products once resolved

3. Event Loop Prediction Drill

Add a checkout button whose click handler mixes console.log, setTimeout(fn, 0), and a fetch() call chained with .then() → predict the exact print order on paper first → run it and compare → write one paragraph on why microtasks always drain before the next macrotask

Study Resources

- MDN: JavaScript Guide
- javascript.info: The Modern JavaScript Tutorial
- MDN: Using Promises · MDN: Microtasks and the JS runtime

TypeScript

JavaScript with a type system layered on top — catching a whole category of bugs before the code ever runs.

Subtopics: basic types & interfaces · union/intersection types · type narrowing · generics

1. Type the ShopFlow Domain

Add Product, CartItem, and Order interfaces matching your Part A Spring Boot DTOs → type every cart function's parameters and return value → fix every compiler error

2. Discriminated Union: Order Status

Model order status as a discriminated union (pending | shipped | delivered | cancelled) sharing a status field → write a status-badge function switching on it → confirm TypeScript narrows correctly in each case → add a new status and let the compiler flag the missing case

3. Strict Mode Migration

Enable strict: true in tsconfig.json across the ShopFlow codebase → fix every new error, paying special attention to implicit any → confirm the project still builds cleanly

Study Resources

- TypeScript Official Handbook
- TypeScript: TSConfig Reference
- Total TypeScript: Beginner's Tutorial

Phase 5 Ship-It Checklist — Testing · Debugging · Accessibility · Production

- **Testing:** HTML validates cleanly (W3C validator); manual cross-browser check in Chrome, Firefox, Safari.
- **Debugging:** comfortable reading a layout with the Elements/Inspector box-model panel and diagnosing a broken Flex/Grid layout without guessing.
- **Accessibility:** every interactive element is keyboard-reachable; landmark regions and heading order are correct; contrast checked with a dev-tools contrast checker.
- **Production:** Tailwind production build purges unused classes; images have explicit width/height to prevent layout shift; portfolio meta tags render correctly in a social-preview debugger.

Final Project — ShopFlow: Static Storefront (Refined)

- Product listing and detail pages in semantic, accessible HTML
- Fully responsive layout (Grid/Flexbox) styled with Tailwind; at least one component rebuilt to respond to a **container query**, not just the viewport
- **NEW:** the cart drawer and a filter panel built with shadcn/ui components instead of hand-rolled markup
- Cart in vanilla TypeScript — add/remove, running total, localStorage persistence, fully typed
- Product data from a local JSON file (stand-in for the real API)
- Companion developer portfolio site, live, showcasing this build
- No framework yet — proves you can build a real, accessible UI with just HTML/CSS/TS

This is your frontend fundamentals proof-of-concept, the same way Phase 1's console cart proved your OOP. Phase 6 doesn't rebuild this — it **migrates and extends** this exact codebase into React, wired to your real Spring Boot API.

PHASE 6 OF 10

Frontend Application Layer

React 19 · Component Architecture & Design Systems · Forms · TanStack Query · State Management · Testing & A11y · Performance · Next.js Primer

Now migrate your Phase 5 storefront into React and connect it to your real Spring Boot backend from Part A. Deployment gets its own full track in Part C — this phase is the application code and the ecosystem tools a real 2026 React codebase actually uses.

Continuity: Topic 1 migrates the Phase 5 static storefront into a React app — it is not rewritten from a blank create-vite template with placeholder content. Every subsequent topic adds one capability (forms, caching, state, tests, performance, SSR) directly onto that same running app.

React 19 + TypeScript

Updated

A component-based UI library, typed with TypeScript, now including React 19's Actions model and the resilience patterns a real app needs beyond the basics.

Subtopics: JSX, props, state, hooks · **React 19: Actions, useActionState, useOptimistic, use(), ref-as-prop [NEW]** · useReducer for complex local state [NEW] · custom hooks [NEW] · React Router (protected routes) · error boundaries [NEW] · focus management in an SPA [NEW]

1. Migrate ShopFlow to React

Scaffold a Vite + React 19 + TypeScript app → port the Phase 5 product listing, detail, and cart markup into typed components, preserving the Tailwind/shadcn styling → confirm visual parity with the static version before adding any new behavior

2. Auth Pages with React 19 Actions

Build Login/Register using useActionState and a form Action instead of manual onSubmit + useState plumbing → call your Phase 3 auth API on submit → store the returned JWT → build a ProtectedRoute wrapper redirecting unauthenticated users

3. Product Listing → Real Data + Custom Hook

Fetch products from your Spring Boot API, replacing the local JSON → add category filter and search → extract the fetch-and-loading logic into a useProducts custom hook → reuse it on a second page and confirm one shared source of truth

4. Error Boundary + Focus Management

Wrap the product routes in an ErrorBoundary with a fallback UI instead of a blank screen → add focus-trapping to the auth modal, returning focus to the trigger button on close → confirm a screen reader announces route changes

Study Resources

- React Official Documentation · React 19 Upgrade Guide
- React Router Documentation
- TypeScript: React Cheatsheet · React: Error Boundaries

Component Architecture & Design Systems

New

How to structure components so the codebase stays maintainable as it grows, and how to formalize the visual language you built in Tailwind into a real design system.

Subtopics: compound components · controlled vs. uncontrolled · prop drilling vs. lifting state up · children-as-function / render props · design tokens as the system's source of truth · when to split a component

1. Compound Component: Product Detail Tabs

Build a Tabs API (specs / reviews / shipping) using React Context internally → the parent manages active-tab state while children just render → compare ergonomics to a single component taking a giant props object

2. Controlled vs. Uncontrolled Audit

Take two ShopFlow form inputs built as controlled components → rebuild one as uncontrolled using a ref (e.g. the checkout file-upload for a receipt) → write 3 bullets on when you'd actually choose uncontrolled

3. ShopFlow Design System — Button, Input, Modal, Table, Pagination

Extract typed Button and Input from your Tailwind/shadcn styles → extract a reusable Modal, Table, and Pagination → promote your Tailwind config tokens (color, spacing, type scale) into a documented token set → replace every ad-hoc styled element across the app with these five components

Study Resources

- React: Passing Data Deeply with Context
- patterns.dev: Compound Pattern
- React: Sharing State Between Components

Forms with React Hook Form + Zod

New

Hand-rolled form state — a useState per field, manual validation — is exactly the repetitive, error-prone work the ecosystem already solved. This is what a real form-heavy PR looks like.

Subtopics: RHF: register, handleSubmit, formState · Zod schema validation · zodResolver integration · field- vs. form-level errors · async validation

1. Checkout Form — the standard RHF + Zod workflow

Define a Zod schema for the real checkout fields (name, email, address, card) → wire it to RHF via zodResolver → render field-level errors → confirm submission is blocked until the schema passes

2. Async Email Availability Check

On the Register form, check email availability against your Spring Boot API as the user types → debounce the check → show pending/valid/invalid state per keystroke pause

3. Reusable FormField Component

Extract a typed FormField combining label + input + error message, using the design-system Input from the previous topic → wire it to RHF's register → use it across checkout and the auth forms so one component renders every form field in the app

Study Resources

- React Hook Form Documentation
- Zod Documentation
- React Hook Form: Zod Resolver

Axios + TanStack Query

Updated

Axios stays for the client/interceptor layer; the fetching and caching pattern is TanStack Query, not manual useEffect. Caching, background refetching, and deduplication are the actual hard parts of data fetching.

Subtopics: Axios centralized instance & interceptors · **TanStack Query: queries & mutations [NEW]** · cache invalidation & refetching [NEW] · optimistic updates [NEW] · JWT attach + auto-refresh on 401

1. Axios API Client Setup

Create one axios.create() instance with baseURL and default headers → request interceptor attaches the JWT automatically → response interceptor handles 401 with auto token refresh

2. Migrate Product Fetching to TanStack Query

Replace the useProducts hook's manual useEffect fetch with useQuery, passing the Axios instance as fetcher → delete the manual loading/error state → confirm a second visit shows cached data instantly while revalidating in the background

3. Optimistic Add-to-Cart Mutation

Wire "add to cart" through useMutation → update the cart UI optimistically before the server responds, using React 19's useOptimistic where it fits → configure onError to roll back → throttle the network in dev tools and watch the optimistic state, then the real confirmation

Study Resources

- Axios Documentation · Axios: Interceptors Guide
- TanStack Query Documentation
- TanStack Query: Optimistic Updates

State Management: Context vs. Zustand New

Separating server state (TanStack Query's job) from client/UI state (Context or Zustand's job) is the distinction most junior React work gets wrong.

Subtopics: server state vs. client state · Context: when it's the right size · Context's re-render cost at scale · Zustand: a minimal external store · choosing per-feature, not project-wide

1. Cart State in Context

Move the cart's quantity controls, remove button, and running total into a Context provider → persist across a page refresh

2. Reproduce Context's Re-render Problem, Then Fix

Add a fast-changing value (live search-input state) into the existing cart Context → watch every consumer re-render on every keystroke via React DevTools' render highlighting → move just that value into a Zustand store → confirm unrelated components stop re-rendering

3. Server vs. Client State Audit

List every piece of state in ShopFlow so far → mark each "server state" (TanStack Query) or "client state" (Context/Zustand/useState) → move any misplaced state to the right layer → write 2 bullets explaining the distinction to a teammate

Study Resources

- React: Passing Data Deeply with Context
- Zustand Documentation
- TkDodo's blog: Practical React Query (server vs. client state)

Types of APIs You'll Work With

Recognizing API styles beyond the REST/GraphQL you've already built.

Subtopics: REST (recap) · GraphQL (recap) · WebSocket/Server-Sent Events · Webhooks · gRPC & SOAP (awareness only)

1. Live Order Status Widget

Connect to a WebSocket/SSE endpoint from React → subscribe to order-status events → update the ShopFlow Order History page in real time → confirm no polling requests are happening

2. Compare Polling vs. WebSocket

Build the same live-status feature with setInterval polling first, then with WebSocket/SSE → compare network-tab traffic on both → write 3 bullets on the trade-offs

Study Resources

- MDN: WebSockets API · MDN: Server-Sent Events
- Spring: WebSocket Support Reference

Testing & Accessibility Audits New

A frontend that ships zero tests, and zero automated accessibility checks, trains habits that don't survive a real team's CI pipeline. This lands right after components are built.

Subtopics: Vitest basics · React Testing Library: query by role, not implementation detail · mocking API calls · **axe-core in component tests (vitest-axe) [NEW]** · one Playwright E2E flow · **@axe-core/playwright automated a11y scans [NEW]** · what to test vs. what not to

1. Component Tests: Design System

Write Vitest + RTL tests for Button, Input, and Modal → query by role/label, not class name or test-id → test disabled state and click handlers → add a vitest-axe assertion to each so a regression that breaks accessible naming fails CI, not just a human reviewer

2. Mocked API Component Test

Test the Product Listing page with the Axios/TanStack Query call mocked → assert the loading skeleton renders first, then the product list → assert the empty-results message renders when the mock returns []

3. Playwright E2E + Automated A11y Scan

Write one Playwright test: log in → add a product to cart → complete checkout → assert the order appears in Order History → run @axe-core/playwright against the checkout and product-detail pages in the same test → fail the test on any serious/critical violation → run headed once, then headless in CI mode

Study Resources

- Vitest Documentation · React Testing Library Documentation
- Playwright Documentation · @axe-core/playwright
- Kent C. Dodds: Common Testing Mistakes

Performance: Code-Splitting, Suspense & Core Web Vitals New

The same "measure before optimizing" instinct already used well in Part A's database-indexing phase, applied to the frontend — profile first, then fix the specific thing that's actually slow.

Subtopics: React.Lazy & Suspense · route-based code-splitting · bundle-size awareness · Lighthouse & Core Web Vitals (LCP, CLS, INP) · when useMemo/useCallback actually help

1. Baseline Lighthouse Pass

Run a Lighthouse audit on ShopFlow as it stands → record LCP, CLS, and INP → identify the single largest JS bundle chunk in the network tab

2. Code-Split Order History with Suspense

Wrap the Order History route in React.Lazy + Suspense, with a skeleton fallback → confirm its JS chunk loads only when visited → rerun Lighthouse and compare initial bundle size before/after

3. Prove Memoization Before You Reach For It

Find a component re-rendering unnecessarily using the React DevTools Profiler → add useMemo/useCallback only where the profiler shows a real cost → remove one "just in case" memoization and confirm no measurable difference → write one paragraph on why memoizing everything by default is itself an anti-pattern

Study Resources

- [React: React.lazy Reference](#) · [React: Suspense](#)
- [web.dev: Core Web Vitals](#) · [Chrome DevTools: Lighthouse](#)
- [React DevTools Profiler](#)

Frontend Dev Tools Updated

The tools you'll reach for constantly while building the above — reference, not a project list.

Subtopics: [npm/pnpm/yarn](#) · [Vite: dev server, HMR, build config](#) · [ESLint + Prettier](#) · [browser DevTools \(Network, Console, Application\)](#) · [React Developer Tools](#) · [Postman/Insomnia](#) · **[Storybook for isolated component development \[NEW\]](#)**

1. Dev Tools Fluency Pass

Inspect a failed ShopFlow API call in the Network tab and identify the exact response body → use the Application tab to inspect the stored JWT and cart persistence → profile a re-render with React DevTools

2. Storybook for the Design System

Add Storybook to the project → write stories for Button and Modal covering default/disabled/loading states → confirm you can develop and visually check a component without running the full app

Study Resources

- [Vite Documentation](#) · [ESLint Documentation](#)
- [Chrome DevTools Documentation](#) · [Storybook Documentation](#)

Next.js Primer New

Enough exposure to SSR, file-based routing, and Server Components to see the alternative to the SPA-only model — one guided page rebuild, not a full migration.

Subtopics: [file-based routing](#) · [Server vs. Client Components](#) · [SSR/SSG basics](#) · [structured data \(JSON-LD\) for product SEO](#) · [why this solves SPA SEO](#) · [when you'd reach for Next.js vs. stay SPA](#)

1. Rebuild Product Listing in Next.js

Create a fresh Next.js app → rebuild just the Product Listing page using the App Router, fetching in a Server Component instead of useQuery → compare: no loading skeleton needed, because the HTML arrives with data already in it

2. Client Component Boundary

Add an "add to cart" button needing interactivity → mark it "use client" → confirm the rest of the page stays server-rendered → write 3 bullets on how you'd decide the client/server boundary on a real page

3. SEO Comparison + Structured Data

View-source the original SPA's Product Listing and note the empty shell → view-source the Next.js rebuild and note the fully-rendered HTML → add JSON-LD product structured data to the page → explain in a comment why this matters for a crawler that doesn't execute JavaScript

Study Resources

- [Next.js Documentation](#) · [Next.js: App Router](#)
- [React: Server Components](#)

Phase 6 Ship-It Checklist — Testing · Debugging · Accessibility · Production

- **Testing:** Vitest/RTL suite green in CI; one Playwright E2E flow covering login → cart → checkout; mocked-network tests for the data layer.
- **Debugging:** comfortable reading a React DevTools Profiler flame chart and an Axios interceptor stack trace to find where a 401 loop or a stale-cache bug actually originates.
- **Accessibility:** automated axe-core scans pass with zero serious/critical violations on checkout, cart, and auth flows; focus management verified with a screen reader, not just visually.
- **Production:** Lighthouse Core Web Vitals recorded before/after code-splitting; JWT refresh and error boundaries verified under throttled/offline network conditions; env-based config for API base URL.

Final Project — ShopFlow: React Frontend, Production-Ready

- The Phase 5 storefront, migrated (not rebuilt) into React 19 + TypeScript, wired to the real Spring Boot API from Part A
- Centralized Axios client with interceptors for JWT attach + auto-refresh on 401
- Product and cart data fetched via TanStack Query instead of manual useEffect, with an optimistic add-to-cart mutation
- Checkout and auth forms built with React Hook Form + Zod
- Server state (TanStack Query) and client state (Context/Zustand) kept deliberately separate
- Live order status via WebSocket/SSE instead of polling
- Design-system component library (Button, Input, Modal, Table, Pagination) used consistently, developed and documented in Storybook
- Full auth flow: register, login, protected routes, logout
- Vitest + RTL component test suite with axe-core assertions, plus one Playwright E2E flow that includes an automated accessibility scan
- At least one route code-split with React.lazy + Suspense, with a before/after Lighthouse Core Web Vitals comparison
- One page rebuilt in the Next.js primer with structured data and a view-source SEO comparison against the SPA version
- Companion developer portfolio site updated with a ShopFlow case study, screenshots, and a link to the live app

Deploying this frontend (and the Spring Boot backend behind it) to a live URL is covered in full in Part C — Deployment & Cloud, right after External APIs.

Part B — Progress Tracker

Phase 5 — Frontend Foundations

☐ HTML
☐ CSS UPDATED
☐ Tailwind CSS & shadcn/ui UPDATED
☐ JavaScript (ES6+) UPDATED
☐ TypeScript
☐ Phase 5 Ship-It Checklist
☐ Final Project: Static Storefront

Phase 6 — Frontend Application Layer

☐ React 19 + TypeScript UPDATED
☐ Component Architecture & Design Systems NEW
☐ Forms: React Hook Form + Zod NEW
☐ Axios + TanStack Query UPDATED
☐ State Management: Context vs. Zustand NEW
☐ Types of APIs You'll Work With
☐ Testing & Accessibility Audits NEW
☐ Performance & Core Web Vitals NEW
☐ Frontend Dev Tools UPDATED
☐ Next.js Primer NEW
☐ Phase 6 Ship-It Checklist
☐ Final Project: React Frontend, Production-Ready